

**UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA**

Dipartimento di Scienze Fisiche, Informatiche e Matematiche

Corso di Laurea in Informatica

**Controllo semaforico adattivo mediante strategie
Max-Pressure: progettazione e valutazione
sperimentale**

Relatore:

Prof. Giacomo Cabri

Candidato:

Jakub Pawel Homoncik

Anno Accademico 2025–2026

Sommario

Il controllo semaforico a tempi fissi non utilizza informazioni sul traffico osservato e può quindi assegnare verde a direzioni poco richieste mentre si formano code su altri accessi. Questa tesi studia strategie di controllo adattivo basate sul principio Max-Pressure, nel quale ogni incrocio seleziona la fase da servire confrontando le code a monte e a valle dei movimenti consentiti. Il lavoro comprende la progettazione di una piattaforma sperimentale riproducibile basata su SUMO e TraCI, l'implementazione di un controller Max-Pressure e di sette estensioni specialistiche, e la costruzione di una pipeline di ablation capace di generare popolazioni, eseguire batch paralleli e produrre automaticamente metriche e report.

La prima valutazione completa considera una rete sintetica Manhattan 8×8 , tre livelli di domanda e cinque seed. I 135 casi sperimentali sono terminati senza errori e con censura nulla. Rispetto al programma semaforico statico nativo, il miglior controller riduce il tempo medio di attesa del 64,26% nel traffico leggero, del 60,09% nel traffico medio e del 53,05% nel traffico elevato. I risultati mostrano inoltre che l'aggiunta di una regola specialistica non produce necessariamente un beneficio: alcune estensioni migliorano soltanto in specifici regimi, altre rimangono inattive oppure introducono un costo superiore al vantaggio ottenuto. Questa osservazione motiva la successiva applicazione alla rete reale di Bologna e lo studio di controller capaci di adattare la propria modalità operativa al regime locale di traffico.

Parole chiave

- Sistemi di trasporto intelligenti;
- Controllo semaforico adattivo;
- Simulazione del traffico urbano;
- Max-Pressure;
- SUMO.

Indice

Sommario	3
Parole chiave	5
1 Introduzione	11
1.1 Contesto e motivazioni	11
1.2 Problema affrontato	12
1.3 Obiettivi della tesi	12
1.4 Contributi del lavoro	13
1.5 Organizzazione della tesi	13
2 Contesto e stato dell'arte	15
2.1 Controllo semaforico urbano	15
2.1.1 Controllo a tempi fissi	15
2.1.2 Controllo attuato e adattivo	16
2.2 Strategie Max-Pressure	16
2.2.1 Principio di funzionamento	16
2.2.2 Pressione dei movimenti e delle fasi	17
2.2.3 Proprietà e limiti del controllo locale	17
2.3 Estensioni del Max-Pressure	18
2.3.1 Costo delle transizioni semaforiche	18
2.3.2 Prevenzione dello spillback	18
2.3.3 Fairness e prevenzione della starvation	19
2.3.4 Gestione dei plotoni di veicoli	19
2.4 Simulazione microscopica del traffico	19
2.4.1 SUMO	19
2.4.2 Interfaccia TraCI	20
2.5 Situazione iniziale del progetto	20
2.5.1 Componenti software già disponibili	20
2.5.2 Limiti della soluzione iniziale	21

2.5.3	Contributi sviluppati nella tesi	21
3	Requisiti e progettazione del sistema	23
3.1	Requisiti funzionali	23
3.2	Requisiti non funzionali	23
3.2.1	Riproducibilità	24
3.2.2	Modularità	24
3.2.3	Osservabilità	24
3.3	Architettura generale	25
3.3.1	Runner delle simulazioni	25
3.3.2	Generazione delle popolazioni	26
3.3.3	Controller semaforici	26
3.3.4	Raccolta e aggregazione delle metriche	26
3.4	Flusso di esecuzione di una simulazione	27
3.5	Scelte progettuali	27
3.5.1	Separazione tra controller e simulatore	27
3.5.2	Configurazione mediante parametri	28
3.5.3	Gestione delle mappe reali e sintetiche	28
4	Implementazione dei controller Max-Pressure	29
4.1	Rappresentazione dei semafori	29
4.1.1	Identificazione delle fasi principali	29
4.1.2	Costruzione dei movimenti	30
4.1.3	Conservazione delle transizioni di sicurezza	30
4.2	Controller Max-Pressure di base	30
4.2.1	Misura delle code	30
4.2.2	Calcolo della pressione	31
4.2.3	Selezione della fase	31
4.2.4	Vincoli di minimo e massimo verde	31
4.3	Macchina a stati del controller	32
4.3.1	Fase attiva e target pendente	32
4.3.2	Gestione di giallo e all-red	32
4.3.3	Recupero dalle transizioni anomale	33
4.4	Lost-Time-Aware	33

4.5	Minimo verde dinamico N_{min}	33
4.6	Fairness	34
4.7	Penalità downstream	34
4.8	Anti-spillback	35
4.9	Platoon extension	35
4.10	Margini e stabilità dello switch	36
4.11	Contatori di attivazione	36
5	Metodologia e piattaforma sperimentale	37
5.1	Obiettivi della valutazione	37
5.2	Pipeline di ablation	37
5.2.1	Generazione dei casi sperimentali	38
5.2.2	Esecuzione parallela	38
5.2.3	Organizzazione delle run	38
5.2.4	Report automatici	39
5.3	Generazione della domanda di traffico	39
5.3.1	Livelli low, medium e high	40
5.3.2	Distribuzione balanced	40
5.3.3	Distribuzione skewed	40
5.3.4	Distribuzione peak	41
5.3.5	Seed e riproducibilità	41
5.4	Profilo di guida human-light	41
5.5	Controller di confronto	42
5.5.1	Fixed program0	42
5.5.2	Fixed tuned	42
5.5.3	Max-Pressure di base	42
5.6	Metriche	43
5.6.1	Tempo di attesa	43
5.6.2	Tempo di viaggio e time loss	43
5.6.3	Velocità media	43
5.6.4	Viaggi completati e throughput	43
5.6.5	Censura	43
5.7	Criteri di confronto	44
5.8	Ambiente hardware e software	44

5.9	Minacce alla validità	44
6	Valutazione su Manhattan 8x8	47
6.1	Motivazione dello scenario sintetico	47
6.2	Topologia della rete	47
6.3	Configurazione sperimentale	48
6.4	Confronto tra Max-Pressure e controllo fisso	49
6.5	Valutazione delle singole estensioni	51
6.5.1	Lost-Time-Aware	51
6.5.2	Nmin dinamico	51
6.5.3	Fairness	52
6.5.4	Penalità downstream	52
6.5.5	Anti-spillback	52
6.5.6	Platoon extension	53
6.5.7	Margini di switch	53
6.6	Analisi dei contatori di attivazione	53
6.7	Confronto complessivo	54
6.8	Indicazioni ricavate per lo scenario reale	54
	Riferimenti bibliografici	57

1 Introduzione

La regolazione semaforica determina come una risorsa limitata, il diritto di passaggio attraverso un incrocio, viene distribuita tra flussi tra loro conflittuali. Una scelta apparentemente locale può propagarsi lungo la rete: un verde troppo breve lascia una coda residua, un verde inutile sottrae tempo agli altri accessi e una coda che raggiunge l'incrocio precedente può impedire movimenti altrimenti non conflittuali. Per questo motivo il controllo dei semafori costituisce un problema centrale nei sistemi di trasporto intelligenti.

Questo capitolo introduce il problema affrontato, gli obiettivi e i contributi del lavoro. La trattazione rimane volutamente ad alto livello; le formulazioni matematiche, le scelte implementative e il protocollo sperimentale vengono sviluppati nei capitoli successivi.

1.1 Contesto e motivazioni

I piani semaforici a tempi fissi sono semplici, prevedibili e poco costosi da gestire. Essi associano a ciascuna fase una durata definita in anticipo e ripetono ciclicamente la stessa sequenza. La loro efficacia dipende tuttavia dalla corrispondenza tra il profilo di domanda usato durante la progettazione e il traffico effettivamente presente. Quando la domanda cambia nello spazio o nel tempo, il piano può continuare a servire una direzione vuota mentre su un altro accesso si accumulano veicoli.

Il controllo adattivo cerca invece di usare osservazioni correnti della rete per modificare il servizio offerto. Tra le strategie decentralizzate, il Max-Pressure è particolarmente interessante perché non richiede una previsione completa della domanda né un unico ottimizzatore centrale. Ogni intersezione confronta le code a monte e a valle dei movimenti consentiti e assegna il verde alla fase con pressione maggiore. La letteratura ha mostrato che questa famiglia di algoritmi possiede importanti proprietà di stabilità e throughput in opportune ipotesi modellistiche [Var13; Won+12; Lev23].

Il passaggio dalla teoria a una simulazione microscopica realistica introduce però costi e vincoli che il modello ideale tende a semplificare: giallo e tempo di sgombero, minimo e massimo verde, capacità finita delle corsie, spillback, arrivi a plotoni e necessità

di non trascurare movimenti deboli. Ne segue che “scegliere sempre la pressione massima” non è, da solo, una specifica sufficiente per un controller utilizzabile.

1.2 Problema affrontato

Il lavoro affronta due problemi collegati. Il primo è ingegneristico: costruire una piattaforma con cui implementare, eseguire e confrontare in modo riproducibile controller semaforici diversi su reti SUMO sintetiche e reali. Il secondo è sperimentale: stabilire in quali condizioni il Max-Pressure di base o le sue estensioni migliorino effettivamente il flusso rispetto a un programma statico.

L'indicatore principale è il tempo di attesa, affiancato da tempo di viaggio, perdita di tempo, velocità, numero di viaggi completati e censura. Il lavoro non persegue obiettivi ambientali: consumi di carburante ed emissioni non sono utilizzati per selezionare o valutare i controller. Tale delimitazione mantiene coerente l'analisi con la domanda di ricerca, cioè massimizzare l'efficienza degli incroci e del flusso veicolare.

Una difficoltà metodologica importante è evitare conclusioni ottenute da un solo seed o da una simulazione interrotta prima che tutti i veicoli completino il viaggio. Un controller può mostrare una bassa attesa media semplicemente perché i veicoli più problematici sono ancora nella rete. Per questo la piattaforma registra esplicitamente la quota di viaggi non conclusi e considera la censura una condizione necessaria per interpretare correttamente le altre metriche.

1.3 Obiettivi della tesi

Gli obiettivi sono i seguenti:

1. implementare un controller Max-Pressure decentralizzato compatibile con le logiche semaforiche presenti nelle mappe SUMO;
2. introdurre estensioni mirate a switching loss, minimo verde dinamico, fairness, capacità downstream, spillback e arrivi a plotoni;
3. realizzare una pipeline di ablation che generi popolazioni riproducibili, esegua casi in parallelo e produca report confrontabili;
4. definire metriche capaci di distinguere un reale miglioramento da un risultato condizionato da viaggi incompleti;

5. valutare separatamente le varianti su una rete Manhattan regolare, prima di affrontare la maggiore eterogeneità della rete di Bologna;
6. ricavare dai risultati indicazioni progettuali per controller capaci di adattarsi a regimi di traffico differenti.

1.4 Contributi del lavoro

Il progetto di partenza forniva il collegamento fondamentale con SUMO, un runner, un generatore di popolazioni e un controller fisso. Su questa base sono stati sviluppati:

- un'implementazione completa del Max-Pressure, comprensiva di macchina a stati per preservare giallo e all-red;
- sette famiglie di estensioni parametriche e relativi contatori di attivazione;
- una rappresentazione uniforme degli scenari sperimentali e dei profili di guida;
- una pipeline batch con preflight, cache delle popolazioni, parallelismo, tracciamento del progresso e report automatici;
- il calcolo esplicito di throughput e censura a partire dai file `tripinfo`;
- una campagna finale su Manhattan 8×8 composta da 135 simulazioni valide, tre livelli di domanda e cinque seed;
- l'estensione successiva della metodologia a distribuzioni spaziali non uniformi e a controller ibridi per la rete reale di Bologna.

Il contributo non consiste quindi soltanto in un algoritmo, ma in un processo sperimentale osservabile. I contatori interni consentono di verificare se una variante ha prodotto un risultato perché il suo meccanismo si è attivato o se, al contrario, è rimasta operativamente identica al Max-Pressure di base.

1.5 Organizzazione della tesi

Il capitolo 2 presenta il controllo semaforico adattivo, il Max-Pressure e gli strumenti di simulazione. Il capitolo 3 definisce requisiti e architettura della piattaforma. Il capitolo 4 descrive l'implementazione del controller e delle estensioni. Il capitolo 5 formalizza il protocollo sperimentale, le popolazioni e le metriche. Il capitolo 6 analizza la campagna Manhattan 8×8 . La parte successiva della tesi applica le indicazioni ricavate alla rete

di Bologna, introduce i controller ibridi sviluppati per la mappa reale e discute i risultati conclusivi.

2 Contesto e stato dell'arte

Per collocare correttamente il contributo è necessario distinguere tre livelli: le strategie di controllo già note in letteratura, gli strumenti software utilizzati e le componenti effettivamente sviluppate durante il tirocinio. Il capitolo procede dal controllo a tempi fissi al Max-Pressure, ne discute le principali estensioni e conclude descrivendo lo stato iniziale del progetto.

2.1 Controllo semaforico urbano

Un'intersezione semaforizzata è descritta da un insieme di movimenti, ad esempio attraversamento rettilineo o svolta, e da un insieme di fasi. Una fase abilita contemporaneamente movimenti compatibili. Il controller deve decidere quanto a lungo mantenere la fase corrente e quando avviare la transizione verso la successiva. La qualità della decisione dipende sia dal criterio di scelta sia dai vincoli fisici: un cambio di fase richiede intervalli durante i quali la capacità utile si riduce.

2.1.1 Controllo a tempi fissi

Nel controllo a tempi fissi la sequenza, la durata delle fasi e l'eventuale offset rispetto agli altri incroci sono definiti a priori. Il principale vantaggio è la prevedibilità. Un piano ben calibrato può coordinare corridoi e creare onde verdi; inoltre non dipende dall'affidabilità di sensori o comunicazioni.

Il limite è la scarsa reattività. Una volta fissato il ciclo, il verde viene assegnato anche in assenza di domanda e non può essere trasferito immediatamente a una coda inattesa. Il confronto sperimentale deve tuttavia essere formulato con cautela: battere un particolare piano statico non equivale a dimostrare la superiorità rispetto a ogni possibile piano ottimizzato. Nella campagna Manhattan il riferimento è il *program0* nativo della mappa, non un piano coordinato ottimizzato a posteriori sui casi di test.

2.1.2 Controllo attuato e adattivo

I controller attuati usano rivelatori per prolungare o terminare una fase sulla base della presenza veicolare. Regole di *gap-out*, ad esempio, chiudono il verde quando l'intervallo tra passaggi supera una soglia [CF12]. I sistemi adattivi operano a un livello più ampio: stimano lo stato corrente e aggiornano durate, cicli o selezione delle fasi. OPAC e SCOOT sono esempi storici di strategie sensibili alla domanda [Gar83; RB91].

Le soluzioni centralizzate possono coordinare una rete, ma richiedono più informazioni, comunicazione e capacità di calcolo. Le soluzioni decentralizzate riducono tale dipendenza e risultano naturalmente scalabili. Il Max-Pressure appartiene a quest'ultima famiglia: l'azione di ciascun semaforo dipende principalmente dalle code sulle corsie incidenti.

2.2 Strategie Max-Pressure

Il Max-Pressure deriva dalle politiche di backpressure impiegate per stabilizzare reti di code. Applicato al traffico, confronta la quantità di veicoli che desidera attraversare l'incrocio con la capacità residua delle corsie riceventi. L'intuizione è servire i movimenti che riducono maggiormente il differenziale di coda senza trasferire indiscriminatamente congestione a valle.

2.2.1 Principio di funzionamento

Sia $\mathcal{M}$ l'insieme dei movimenti di un semaforo. Un movimento $m = (i, j)$ collega una corsia in ingresso i a una corsia in uscita j . Nella forma adottata nel progetto, la pressione del movimento è

$$p_{ij}(t) = q_i(t) - q_j(t), \quad (2.1)$$

in cui q_i e q_j sono le code osservate all'istante t . Se $\mathcal{M}_\phi$ è l'insieme dei movimenti abilitati dalla fase principale ϕ , la pressione della fase è

$$P_\phi(t) = \sum_{(i,j) \in \mathcal{M}_\phi} p_{ij}(t). \quad (2.2)$$

Il candidato naturale è quindi

$$\phi^*(t) = \arg \max_{\phi \in \Phi} P_{\phi}(t). \quad (2.3)$$

La sottrazione della coda downstream distingue il Max-Pressure da una politica che osserva soltanto gli ingressi: un movimento perde priorità se conduce a una corsia già occupata. Formulazioni più generali includono turning ratio, pesi o portate di saturazione. La scelta semplificata adottata è coerente con l'obiettivo di ottenere un controller locale interpretabile e applicabile alle mappe disponibili.

2.2.2 Pressione dei movimenti e delle fasi

La pressione non rappresenta la sola lunghezza della coda. Una fase può avere molti veicoli in ingresso ma una pressione modesta quando anche le uscite sono congestionate. Al contrario, una coda più corta può risultare prioritaria se può essere scaricata verso corsie libere. Il punteggio di fase somma i movimenti consentiti; ciò permette di valutare con un unico valore configurazioni semaforiche che servono più approcci contemporaneamente.

In una rete regolare il calcolo è completamente locale e può essere ripetuto a ogni passo. Le proprietà teoriche di throughput del Max-Pressure sono state studiate in reti di intersezioni e in implementazioni decentralizzate [Var13; Won+12; Le+15]. Studi di simulazione mostrano tuttavia che parametri, sensori e struttura della rete influenzano le prestazioni osservate [SY18; MUH20].

2.2.3 Proprietà e limiti del controllo locale

L'uso di informazioni locali rende l'algoritmo scalabile e robusto alla mancanza di un coordinatore centrale. Una variazione di domanda modifica immediatamente i differenziali di coda, senza richiedere la stima preventiva di un intero profilo giornaliero.

Esistono però quattro limiti principali. Primo, il criterio ideale non attribuisce automaticamente un costo al cambio di fase. Secondo, le code misurate non descrivono sempre la capacità fisica residua di una corsia. Terzo, massimizzare il servizio complessivo può ritardare ripetutamente un movimento debole. Quarto, decisioni localmente corrette possono non produrre coordinamento lungo un corridoio. La revisione di Levin evidenzia proprio la distanza tra proprietà teoriche e dettagli necessari per implementazioni pratiche [Lev23].

2.3 Estensioni del Max-Pressure

Le varianti studiate nel progetto non sono controller indipendenti dal punto di vista concettuale. Ognuna modifica il punteggio o il momento della decisione per affrontare un limite specifico. Questo rende possibile una valutazione per ablation: si mantiene il nucleo Max-Pressure e si osserva il contributo della singola estensione.

2.3.1 Costo delle transizioni semaforiche

Durante giallo e all-red nessuna delle direzioni principali utilizza pienamente l'intersezione. Cambi troppo frequenti possono quindi ridurre la capacità nonostante ogni scelta sia localmente giustificata. Le strategie *lost-time-aware* introducono una soglia: la nuova fase deve offrire un guadagno sufficiente a compensare il servizio perso durante la transizione. La considerazione esplicita dello switching loss è un tema rilevante anche in formulazioni apprese del Max-Pressure [Wan+22].

Un minimo verde assolve una funzione simile, ma in modo temporale: impedisce di rivalutare immediatamente la fase appena attivata. Un massimo verde costituisce invece una salvaguardia contro permanenze eccessive e forza periodicamente una nuova decisione.

2.3.2 Prevenzione dello spillback

Lo spillback si verifica quando la coda occupa la corsia fino a bloccare l'intersezione a monte. In questa condizione il solo conteggio dei veicoli fermi può reagire troppo tardi o non distinguere correttamente la capacità residua. Le strategie capacity-aware usano occupazione, lunghezza della corsia o stime di capacità per evitare di inviare veicoli verso un'uscita quasi piena [Gre+15; Noa+21].

Nel progetto vengono valutate due forme. La penalità downstream riduce in modo continuo il punteggio dei movimenti diretti verso corsie occupate. Il blocco anti-spillback è invece una protezione discreta: oltre una soglia di attivazione una fase viene esclusa, e torna disponibile soltanto sotto una soglia di rilascio. L'isteresi evita oscillazioni quando l'occupazione si trova vicino al limite.

2.3.3 Fairness e prevenzione della starvation

Una politica orientata al throughput può continuare a favorire flussi forti. Per limitare la starvation, al punteggio di una fase può essere aggiunto un bonus crescente con il tempo trascorso dall'ultimo servizio. Il bonus non sostituisce la pressione, ma rende progressivamente più competitivo un movimento trascurato. Il compromesso tra throughput e fairness è discusso anche da Wu, Liu e Gettman [WLG18].

2.3.4 Gestione dei plotoni di veicoli

I veicoli non raggiungono necessariamente l'incrocio in modo uniforme. Il rilascio di un semaforo a monte può generare un plotone, cioè una sequenza di passaggi con headway ridotto. Interrompere il verde a metà del plotone spreca parte dell'opportunità di servizio e può ricreare rapidamente una coda. La platoon extension osserva gli intervalli di passaggio vicino alla stop line e, entro un limite massimo, prolunga la fase quando il flusso rimane compatto. Questa logica è affine ai criteri di gap-out dei controller attuati, ma viene integrata nella decisione Max-Pressure.

2.4 Simulazione microscopica del traffico

La valutazione di un controller richiede di rappresentare non soltanto flussi aggregati, ma anche partenze, accelerazioni, cambi di corsia e interazioni tra veicoli. Per questo il progetto impiega una simulazione microscopica.

2.4.1 SUMO

SUMO, *Simulation of Urban MObility*, è un simulatore open source, microscopico e a passi discreti, progettato per reti stradali di dimensione urbana [Beh+11; Ecl26a]. Le reti sono descritte tramite file XML contenenti archi, corsie, connessioni, incroci e programmi semaforici. La simulazione produce informazioni per veicolo e consente di eseguire esperimenti senza interfaccia grafica, requisito essenziale per campagne batch.

Nel progetto SUMO gestisce la dinamica veicolare e la validità delle transizioni semaforiche. Il codice Python non sostituisce il simulatore: ne modifica le azioni attraverso un'interfaccia di controllo e raccoglie le osservazioni necessarie.

2.4.2 Interfaccia TraCI

TraCI, *Traffic Control Interface*, espone un protocollo con cui un processo esterno può interrogare e comandare una simulazione SUMO [Weg+08; Ecl26b]. A ogni passo il controller legge, tra le altre grandezze, veicoli fermi, occupazione delle corsie, stato delle fasi e passaggi su rivelatori virtuali. Può quindi richiedere il passaggio a una fase o a un programma semaforico differente.

Questa separazione è utile scientificamente: il modello del traffico rimane quello di SUMO, mentre la politica di controllo è implementata in Python e può essere modificata senza rigenerare la rete.

2.5 Situazione iniziale del progetto

Le linee guida di tesi richiedono di distinguere quanto già disponibile da quanto realizzato nel lavoro. La ricostruzione è stata effettuata attraverso la cronologia Git e il confronto tra il commit iniziale applicativo e la versione finale.

2.5.1 Componenti software già disponibili

La base iniziale comprendeva:

- un runner capace di avviare SUMO e avanzare la simulazione;
- la classe astratta comune ai controller;
- un controller a tempi fissi;
- un generatore di popolazioni YAML;
- strutture per veicoli, tipi e metriche elementari;
- utility per risolvere i percorsi delle mappe;
- una raccolta di reti sintetiche e reali, tra cui Manhattan e Bologna.

Era quindi già presente l'infrastruttura minima per inserire veicoli e interagire con i semafori. Non erano invece disponibili un Max-Pressure completo, le varianti specialistiche, una campagna automatizzata di ablation e la verifica esplicita dei viaggi non conclusi.

2.5.2 Limiti della soluzione iniziale

L'esecuzione di una singola simulazione non è sufficiente per un confronto sperimentale. Mancavano una matrice dichiarativa dei casi, isolamento degli output, esecuzione parallela, controllo preventivo delle configurazioni, aggregazione su seed e report delle differenze rispetto a una baseline. La sola media sui veicoli arrivati, inoltre, poteva nascondere un'elevata censura.

Dal lato del controllo, un template intelligente non risolveva il problema della mappatura tra segnali SUMO, corsie e movimenti, né la gestione sicura di giallo e all-red. Questi aspetti sono risultati più complessi del calcolo della sola pressione e hanno richiesto una macchina a stati esplicita.

2.5.3 Contributi sviluppati nella tesi

L'evoluzione del repository mostra una progressione incrementale: prima il Max-Pressure puro, quindi spillback, lost-time-aware, fairness, Nmin, penalità downstream e plotoni; successivamente la pipeline di ablation, la correzione delle metriche e i controller ibridi per Bologna. La scelta di aggiungere una funzione alla volta ha permesso di attribuire i risultati e di individuare estensioni inattive o controproducenti. I capitoli seguenti descrivono questa architettura e l'implementazione risultante.

3 Requisiti e progettazione del sistema

Un esperimento sui controller semaforici è credibile soltanto se la piattaforma impedisce confronti non omogenei e conserva abbastanza informazioni per riprodurre ogni risultato. Da questa esigenza derivano i requisiti e l'architettura descritti nel capitolo. La progettazione separa la politica di controllo dalla generazione del traffico e dalla valutazione, in modo che una modifica a un componente non alteri implicitamente gli altri.

3.1 Requisiti funzionali

Il sistema deve permettere di:

1. selezionare una mappa SUMO e una popolazione di veicoli;
2. eseguire sia il programma semaforico nativo sia un controller Max-Pressure configurabile;
3. associare a ogni variante un insieme esplicito di parametri;
4. generare più livelli e distribuzioni di domanda usando seed noti;
5. avanzare la simulazione fino all'esaurimento dei viaggi o a un limite dichiarato;
6. salvare metriche per veicolo, metriche aggregate e contatori interni del controller;
7. costruire automaticamente il prodotto cartesiano tra mappe, domande, seed e scenari;
8. eseguire più casi in parallelo senza collisioni tra file;
9. produrre classifiche e differenze rispetto a una baseline scelta.

Un requisito ulteriore, emerso durante lo sviluppo, è la capacità di rilevare errori prima di avviare campagne lunghe. La fase di preflight deve quindi controllare nomi delle mappe, scenari, file di rete, opzioni dei controller e percorsi di output.

3.2 Requisiti non funzionali

Oltre al risultato numerico, sono rilevanti riproducibilità, modularità e osservabilità. Questi requisiti hanno guidato scelte quali la serializzazione delle popolazioni, l'uso di configurazioni risolte e l'introduzione di contatori per le regole specialistiche.

3.2.1 Riproducibilità

Due controller devono essere confrontati sugli stessi veicoli, con uguali route, istanti di partenza e caratteristiche di guida. Per ogni coppia mappa–domanda–seed viene quindi generato un unico file di popolazione, riutilizzato da tutti gli scenari. Il seed compare sia nel nome sia nel contenuto del file. La directory di una campagna conserva inoltre la configurazione effettivamente risolta, evitando che un valore predefinito modificato in seguito renda ambiguo un risultato precedente.

La riproducibilità non implica che seed diversi producano lo stesso valore; serve piuttosto a rendere controllata la variabilità. Nella valutazione finale Manhattan ogni scenario viene eseguito sui medesimi cinque seed, consentendo confronti appaiati.

3.2.2 Modularità

Il simulatore, il controller, il generatore della domanda e la reportistica hanno responsabilità distinte. Tutti i controller rispettano un’interfaccia comune di collegamento e aggiornamento; il runner non deve conoscere la formula specifica con cui viene scelta una fase. Analogamente, la pipeline batch costruisce comandi per una simulazione singola senza duplicarne la logica.

Le estensioni del Max-Pressure sono abilitate da flag e parametri. Questo consente di mantenere un solo nucleo implementativo e di definire scenari sperimentali come configurazioni, anziché creare copie divergenti del controller.

3.2.3 Osservabilità

Una differenza nelle metriche finali non rivela da sola il comportamento che l’ha prodotta. Il sistema registra quindi numero di switch, attivazioni del massimo verde, passi di hold Nmin, eventi di blocco e rilascio spillback, prolungamenti per plotone e bonus di fairness. Se una variante coincide numericamente con il controller base e il suo contatore rimane a zero, è possibile concludere che il meccanismo non è stato sollecitato, non che abbia fallito dopo essersi attivato.

Ogni worker scrive inoltre un log separato. Il file di progresso aggrega casi completati, riusciti e falliti, rendendo possibile controllare una campagna eseguita in background.

3.3 Architettura generale

La figura 3.1 mostra i componenti principali. La pipeline di ablation genera la matrice dei casi e usa il runner per avviare processi SUMO indipendenti. Il runner carica la popolazione, collega il controller mediante TraCI e inoltra le osservazioni al modulo delle metriche. Al termine, gli output dei casi vengono aggregati in report di campagna.

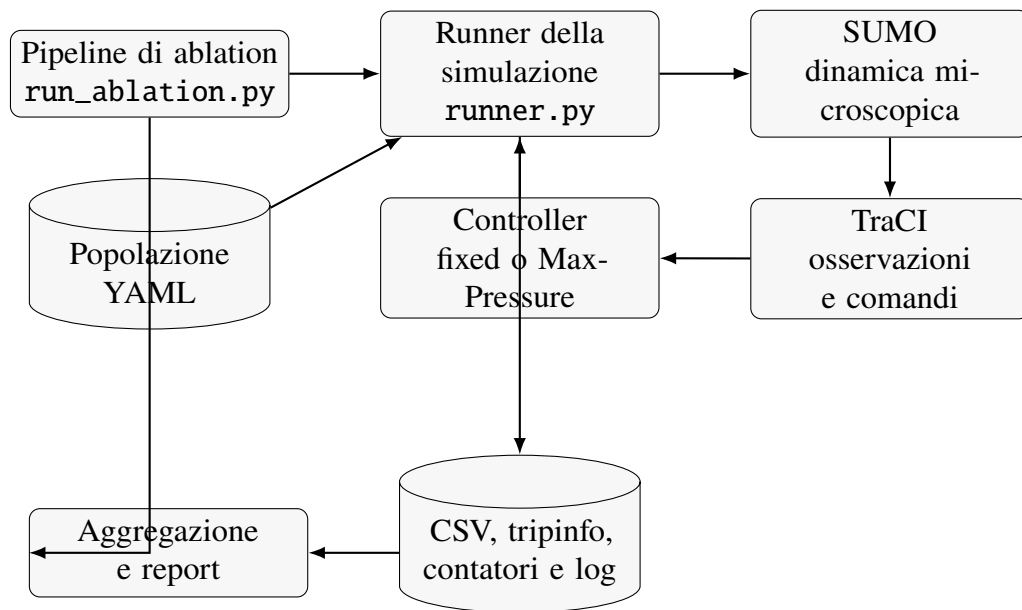


Figura 3.1. Architettura della piattaforma sperimentale.

3.3.1 Runner delle simulazioni

`runner.py` è il punto di ingresso per un singolo caso. Il modulo interpreta gli argomenti, individua i file della mappa, avvia SUMO, carica la popolazione e istanzia il controller richiesto. Il ciclo principale continua finché `getMinExpectedNumber()` segnala veicoli presenti o ancora attesi, oppure finché viene raggiunto il limite opzionale di passi.

A ogni iterazione il runner avanza SUMO e invoca l'aggiornamento del controller. Alla chiusura legge il file `tripinfo`, combina le informazioni di viaggio con i contatori online e scrive i risultati del caso. Questa sequenza centralizza il ciclo di vita della simulazione e garantisce che `fixed` e `Max-Pressure` differiscano soltanto nella politica semaforica.

3.3.2 Generazione delle popolazioni

`generate_population.py` seleziona route valide dalla mappa e genera veicoli con istante di partenza, tipo e parametri di guida. Il risultato è un file YAML leggibile e riutilizzabile. `src/population.py` traduce poi questa descrizione in comandi TraCI durante l'esecuzione.

La separazione tra generazione e simulazione impedisce che due scenari consumino in ordine diverso lo stesso generatore casuale. Il traffico non viene rigenerato per ogni controller: una data popolazione è un input immutabile del confronto.

3.3.3 Controller semaforici

La classe base definisce il contratto comune. Il controller fixed può lasciare in esecuzione un programma esistente o modificarne selettivamente il verde. Il controller Max-Pressure costruisce, per ogni semaforo, una rappresentazione di fasi principali, movimenti, corsie e stato temporale. Tutti i semafori della mappa vengono collegati all'avvio e aggiornati in modo decentralizzato.

La configurazione del Max-Pressure è parametrica. Gli scenari non contengono codice, ma combinazioni di flag quali `-lost-time-aware`, `-nmin-dynamic` o `-spillback-block`. Questa scelta ha reso possibile correggere il nucleo una sola volta e confrontare varianti che condividono la medesima gestione delle transizioni.

3.3.4 Raccolta e aggregazione delle metriche

Le metriche online descrivono lo stato e le decisioni del controller; quelle post-simulazione provengono dai viaggi completati. Per ciascun caso vengono salvati un CSV per veicolo, un riepilogo e i log. La pipeline combina poi i casi in:

- `run_results.csv`, una riga per simulazione;
- `summary_by_group.csv`, statistiche aggregate per scenario;
- `summary_vs_base.csv`, differenze rispetto alla baseline;
- `summary.md`, tabella leggibile della campagna;
- `winners.txt` e `table.txt`, classifiche sintetiche;
- `config_resolved.yaml`, configurazione effettiva.

Sebbene il formato generico possa contenere campi prodotti da SUMO per altre analisi, la tesi aggrega e interpreta esclusivamente metriche di mobilità e completezza dei viaggi.

3.4 Flusso di esecuzione di una simulazione

Il flusso completo è il seguente:

1. la pipeline risolve mappe, scenari, livelli di domanda e seed;
2. per ogni combinazione mappa–distribuzione–domanda–seed genera o recupera dalla cache una popolazione;
3. un worker prepara una directory isolata e avvia il runner;
4. il runner avvia un processo SUMO con i file della mappa;
5. i veicoli vengono inseriti secondo il file YAML;
6. il controller analizza i programmi semaforici e costruisce il proprio stato interno;
7. a ogni passo SUMO aggiorna veicoli e corsie, quindi il controller osserva lo stato e può richiedere una nuova fase;
8. al termine vengono chiusi TraCI e SUMO e viene analizzato il `tripinfo`;
9. il worker registra successo o fallimento e aggiorna il progresso;
10. completati tutti i casi, i risultati vengono raggruppati per mappa, distribuzione, domanda e scenario.

La generazione delle popolazioni precede l'esecuzione parallela. In questo modo nessun worker modifica un input condiviso mentre altri casi sono in corso.

3.5 Scelte progettuali

3.5.1 Separazione tra controller e simulatore

Si è scelto di non incorporare la logica adattiva nei file XML dei programmi semaforici. Il controller Python osserva e comanda SUMO mediante TraCI. Il vantaggio è la facilità di sperimentazione e di strumentazione; lo svantaggio è un costo di comunicazione a ogni passo. Per le dimensioni studiate, la maggiore chiarezza e modificabilità prevalgono su tale costo.

La separazione consente inoltre di preservare le transizioni definite nella mappa. Il controller sceglie una fase principale target, ma non sintetizza arbitrariamente una stringa di segnali potenzialmente insicura.

3.5.2 Configurazione mediante parametri

Le varianti sono definite in uno *scenario pack*. Ogni scenario associa un nome stabile al controller e ai flag. La configurazione dichiarativa riduce gli errori nei comandi lunghi, permette di riportare esattamente i parametri nei risultati e rende semplice escludere combinazioni non pertinenti. Per la campagna Manhattan sono state scelte una variante rappresentativa per ciascuna famiglia, anziché tutte le combinazioni esplorate durante il tuning.

3.5.3 Gestione delle mappe reali e sintetiche

Le mappe sintetiche hanno topologia regolare e sono adatte a isolare il comportamento algoritmico. Le mappe reali contengono numeri diversi di corsie, fasi eterogenee, svolte, segmenti brevi e piani semaforici specifici. Il layer di risoluzione dei percorsi e l'analisi dinamica dei programmi evitano di codificare nel controller gli identificativi di una singola rete.

Non tutte le strategie sono tuttavia trasferibili senza adattamento. Un programma statico specifico della mappa, come il `program0` di Bologna, non ha un equivalente semantico universale. Per questo i capitoli sperimentali separano la valutazione generale su Manhattan dagli sviluppi specifici della rete reale.

4 Implementazione dei controller Max-Pressure

Il calcolo della pressione occupa poche righe; la parte più delicata consiste nel trasformare quel calcolo in azioni compatibili con programmi semaforici eterogenei. Il capitolo descrive prima la rappresentazione dei semafori e la macchina a stati, quindi le estensioni valutate. Le formule corrispondono alla versione effettivamente utilizzata nella campagna Manhattan.

4.1 Rappresentazione dei semafori

All'avvio il controller interroga TraCI per ottenere tutti i semafori, le logiche disponibili e i link controllati. Per ciascun semaforo costruisce una struttura contenente numero e durata delle fasi, fasi principali, movimenti per fase, tempi di attesa interni e stato della transizione. La costruzione non presuppone identificativi specifici della mappa.

Quando più logiche sono disponibili viene scelta quella con il maggior numero di fasi principali, preferendo il programma già attivo se appartiene ai candidati migliori. Questa regola evita di imporre universalmente un identificativo, ad esempio 1, che su una mappa reale potrebbe avere un significato differente.

4.1.1 Identificazione delle fasi principali

Una fase è considerata principale se la stringa di stato contiene almeno un segnale verde, g o G, e non contiene segnali gialli. Le fasi solo gialle o rosse sono trattate come transizioni. Questa distinzione è necessaria perché il Max-Pressure deve scegliere tra configurazioni di servizio, non tra gli stati intermedi richiesti per passare in sicurezza da una configurazione all'altra.

La classificazione è ricavata dalla logica SUMO e non da una lista scritta a mano. Ciò permette allo stesso controller di operare su incroci con numeri di fasi e lunghezze delle stringhe di segnale diversi.

4.1.2 Costruzione dei movimenti

`getControlledLinks()` associa ogni posizione della stringa semaforica a uno o più collegamenti tra corsia in ingresso e corsia in uscita. Per ogni fase principale il controller visita le posizioni verdi e costruisce l'insieme

$$\mathcal{M}_\phi = \{(i, j) \mid \text{il movimento } i \rightarrow j \text{ è verde nella fase } \phi\}. \quad (4.1)$$

Eventuali duplicati vengono eliminati. Il risultato è una relazione esplicita tra una fase e le corsie sulle quali misurare code, occupazione e passaggi. Se una fase non abilita alcun movimento valido non partecipa alla selezione.

4.1.3 Conservazione delle transizioni di sicurezza

Il controller non passa direttamente da una stringa verde a un'altra. Quando seleziona un target, avanza alla fase successiva prevista dalla logica e memorizza il target come `pending_target`. Durante giallo e all-red attende la durata configurata nel programma SUMO. Se la sequenza raggiunge una fase principale diversa dal target, effettua il riallineamento soltanto al termine della transizione.

Questa soluzione conserva le durate di sicurezza della mappa e separa due decisioni: *quale* fase servire e *come* raggiungerla. Un meccanismo di recupero interviene se il semaforo rimane per almeno 20 secondi in uno stato non principale, condizione considerata anomala. Tale salvaguardia è stata introdotta dopo aver osservato stalli in campagne iniziali.

4.2 Controller Max-Pressure di base

4.2.1 Misura delle code

La coda su una corsia è misurata mediante `getLastStepHaltingNumber()`, cioè il numero di veicoli considerati fermi nell'ultimo passo. La misura viene memorizzata in una cache valida per il passo corrente, perché la stessa corsia può comparire in più movimenti. Si indica con $q_l(t)$ il valore associato alla corsia l .

La scelta del numero di veicoli fermi, anziché del numero totale presente, rende il punteggio sensibile alla congestione effettiva vicino all'intersezione. Non rappresenta

tuttavia in modo completo veicoli in avvicinamento o capacità fisica residua; le estensioni downstream e platoon sono state introdotte anche per compensare questa limitazione.

4.2.2 Calcolo della pressione

Per ogni movimento (i, j) si applica l'equazione (2.1). La pressione della fase è la somma dell'equazione (2.2). In assenza di estensioni, l'algoritmo produce inoltre la domanda della fase

$$D_\phi(t) = \sum_{(i,j) \in \mathcal{M}_\phi} q_i(t), \quad (4.2)$$

utilizzata dal minimo verde dinamico e, nei controller successivi, per classificare il regime di traffico.

Una pressione negativa è ammessa: indica che le uscite sono più cariche degli ingressi. Se tutte le fasi valide hanno un punteggio finito, viene comunque scelta la migliore in senso relativo. Nel caso anti-spillback una fase può ricevere valore $-\infty$ e viene esclusa.

4.2.3 Selezione della fase

Siano S_ϕ i punteggi, uguali alla pressione salvo bonus o penalità. La fase migliore cambia il semaforo soltanto se

$$\phi^* \neq \phi_c \quad \wedge \quad S_{\phi^*} > S_{\phi_c} + M, \quad (4.3)$$

in cui ϕ_c è la fase corrente e M il margine di switch. Nel controller base $M = 0$. La disuguaglianza stretta evita cambi in caso di pareggio. Se la condizione è soddisfatta, il controller registra lo switch, imposta il target pendente e avvia la sequenza di transizione.

4.2.4 Vincoli di minimo e massimo verde

Il valore predefinito del minimo verde è 10 secondi. Prima di tale durata la fase non viene rivalutata. Il massimo verde è 120 secondi e opera come safety cap: se esiste una fase alternativa finita, il controller può forzare il cambio quando il limite è raggiunto. Minimo e massimo non determinano quindi un ciclo fisso; delimitano la frequenza delle decisioni adattive.

È importante distinguere questo comportamento da un piano statico. Il fixed mantiene una durata prestabilita indipendentemente dalle code. Il Max-Pressure mantiene la fase almeno per il minimo, poi può conservarla o cambiarla in base ai punteggi osservati.

4.3 Macchina a stati del controller

Per ogni semaforo vengono mantenuti fase principale attiva, target pendente, tempo di permanenza in stati non principali, hold Nmin, tempo di svuotamento e prolungamento per plotone. Queste variabili rendono il controller una macchina a stati, non una funzione priva di memoria.

4.3.1 Fase attiva e target pendente

Nello stato ordinario il semaforo si trova in una fase principale. Il controller misura la rete e può restare oppure scegliere un target. Quando sceglie un target diverso, entra nello stato di transizione e non prende nuove decisioni di pressione fino al completamento della sequenza. In questo modo un punteggio che cambia durante il giallo non interrompe o inverte la manovra.

Raggiunto il target, lo stato attivo viene aggiornato e i temporizzatori della nuova fase vengono inizializzati. I tempi di attesa delle fasi non servite sono mantenuti soltanto quando è abilitata la fairness.

4.3.2 Gestione di giallo e all-red

La durata persa nel lasciare una fase è calcolata sommando le durate delle fasi non principali che la seguono fino alla prossima fase principale:

$$L_{\phi_c} = \sum_{k \in \mathcal{T}(\phi_c)} d_k, \quad (4.4)$$

con $\mathcal{T}(\phi_c)$ insieme degli stati di transizione e d_k relativa durata. Il valore viene riutilizzato sia dal margine lost-time-aware sia dal calcolo Nmin. Non viene quindi assunto un giallo uguale per tutti gli incroci.

4.3.3 Recupero dalle transizioni anomale

Due controlli evitano stalli. Se esiste un target pendente e una fase di transizione termina, il controller avanza o si riallinea al target. Se invece non esiste un target ma il semaforo rimane in una fase non principale oltre la soglia di 20 secondi, viene ripristinata la prima fase principale valida. Il recupero non è una politica di traffico: è una protezione operativa contro stati incoerenti o logiche di rete non previste.

4.4 Lost-Time-Aware

La variante lost-time-aware richiede che il guadagno di punteggio superi una stima del costo della transizione. Il margine complessivo è

$$M = \varepsilon_{abs} + \varepsilon_{rel}|S_{\phi_c}| + \gamma s L_{\phi_c}, \quad (4.5)$$

in cui γ è il guadagno lost-time, s una portata di saturazione espressa in veicoli al secondo e L_{ϕ_c} il tempo perso. La variante finale Manhattan usa $\gamma = 0,60$ e $s = 0,50$.

L'effetto atteso è ridurre switch marginali. Un valore eccessivo può tuttavia rendere il controller lento a reagire e prolungare fasi non più ottimali. Per questo l'efficacia non può essere dedotta dalla sola plausibilità della regola: deve essere misurata insieme al numero di switch e agli interventi del massimo verde.

4.5 Minimo verde dinamico N_{min}

N_{min} stima quanti veicoli sia opportuno servire prima di abbandonare una fase. Dato il costo di transizione dell'equazione (4.4), calcola

$$N_{cost} = \alpha s L_{\phi_c}, \quad (4.6)$$

$$N_{target} = \max\{N_{floor}, N_{cost}, g D_{\phi_c}\}, \quad (4.7)$$

in cui α pesa il costo, N_{floor} è un minimo di veicoli, g il coefficiente dipendente dalla domanda e D_{ϕ_c} la domanda della fase. Se la domanda è positiva, N_{target} viene limitato a non superarla. La durata di hold è

$$H_{\phi_c} = \max \left\{ G_{min}^N, \frac{N_{target}}{s} \right\}. \quad (4.8)$$

Se la fase si svuota prima del termine, un timer consente il rilascio anticipato invece di attendere inutilmente. La configurazione studiata usa $\alpha = 0,80$, $N_{floor} = 2$, $G_{min}^N = 4$ secondi, $g = 0,25$ e rilascio dopo un secondo di domanda nulla.

Questa configurazione modifica due aspetti: abilita l'hold dinamico e riduce da 10 a 4 secondi il minimo verde di base. Nell'interpretare i risultati occorre quindi evitare di attribuire automaticamente ogni guadagno alla formula N_{min} . I contatori mostrano, infatti, che nel traffico basso l'hold dinamico non viene quasi mai attivato; il beneficio deriva soprattutto dalla maggiore rapidità di rivalutazione.

4.6 Fairness

Per ogni fase non servita che presenta domanda viene incrementato un tempo di attesa interno w_ϕ . Quando la fase è attiva, o quando la sua domanda si annulla, il valore viene azzerato. Il bonus è

$$B_\phi(w_\phi) = \mu \frac{w_\phi}{w_\phi + w_{1/2}}, \quad (4.9)$$

ed è aggiunto alla pressione. μ è il bonus asintotico e $w_{1/2}$ il tempo al quale viene raggiunta metà del bonus massimo. La configurazione Manhattan usa $\mu = 5$ e $w_{1/2} = 20$ secondi.

La funzione cresce rapidamente all'inizio e poi satura. Non garantisce un ordine rigido delle fasi, ma rende progressivamente più difficile ignorare un movimento con domanda persistente. Il contatore registra quante volte è stato applicato un bonus positivo e la somma complessiva dei bonus.

4.7 Penalità downstream

La pressione di base sottrae già la coda a valle. La variante downstream aggiunge una penalità continua basata sull'occupazione filtrata con media mobile esponenziale:

$$\bar{o}_j(t) = \eta o_j(t) + (1 - \eta) \bar{o}_j(t - 1), \quad (4.10)$$

$$p'_{ij}(t) = q_i(t) - q_j(t) - \beta \bar{o}_j(t). \quad (4.11)$$

La campagna usa $\beta = 0,4$ e $\eta = 0,30$. Il filtro riduce la sensibilità a picchi di un solo passo. La penalità è morbida: una corsia occupata rende il movimento meno attraente, ma non lo esclude. Il rischio è penalizzare anche una normale occupazione in reti lontane dalla saturazione, introducendo un costo senza prevenire alcun blocco reale.

4.8 Anti-spillback

Il blocco hard usa due stime della saturazione downstream. La prima è l'occupazione filtrata $\bar{o}_j$. La seconda approssima il riempimento della capacità di stoccaggio:

$$f_j = \min \left\{ 1, \frac{q_j}{\max(1, \ell_j/7,5)} \right\}, \quad z_j = \max\{\bar{o}_j, f_j\}, \quad (4.12)$$

in cui ℓ_j è la lunghezza della corsia in metri e 7,5 metri è lo spazio medio di jam assunto per veicolo. Una corsia non bloccata entra nello stato di blocco quando $z_j \geq \theta_{on}$ e sono presenti almeno h_{min} veicoli fermi; rimane bloccata finché $z_j \geq \theta_{off}$. Poiché $\theta_{off} < \theta_{on}$, la regola possiede isteresi.

Nella configurazione `on90_off80`, $\theta_{on} = 0,90$, $\theta_{off} = 0,80$ e $h_{min} = 3$. Se anche un solo movimento di una fase conduce a una corsia bloccata, l'intera fase riceve pressione $-\infty$. È una scelta conservativa: protegge la rete dal trasferimento di congestione, ma può sottrarre servizio anche agli altri movimenti della stessa fase. I contatori di blocco, rilascio e passi bloccati servono a verificare se la protezione sia mai entrata in funzione.

4.9 Platoon extension

Per ogni corsia in ingresso alla fase attiva viene posto un rivelatore virtuale a 8 metri dalla fine. Il controller registra gli istanti in cui i veicoli attraversano tale posizione e calcola la media degli ultimi quattro headway. Un plotone è riconosciuto se la media non supera la soglia configurata e l'ultimo passaggio non è più vecchio del tempo di gap-out.

Quando il controller starebbe per rivalutare la fase, la presenza del plotone può prolungare il verde fino a un massimo aggiuntivo. La configurazione finale usa soglia e gap-out di 2,1 secondi e un massimo di 4 secondi. Un guard downstream impedisce l'estensione solo quando sono contemporaneamente abilitate le misure di spillback o penalità downstream. Nello scenario standalone `mp_platoon_x4` tali moduli non sono attivi; di conseguenza il parametro di guardia dichiarato non modifica il comportamento. Questo dettaglio implementativo è rilevante per non sovrainterpretare il test.

4.10 Margini e stabilità dello switch

Oltre al termine lost-time, il controller supporta un margine assoluto e uno relativo. Con la sola componente relativa,

$$M = \varepsilon_{rel} |S_{\phi_c}|. \quad (4.13)$$

Lo scenario Manhattan `mp_switch_rel005` usa $\varepsilon_{rel} = 0,05$. L'obiettivo è introdurre una banda morta: una fase alternativa deve migliorare il punteggio corrente di almeno il 5%. Anche qui il vantaggio potenziale, meno oscillazioni, si contrappone al rischio di ritardare una decisione utile.

4.11 Contatori di attivazione

Il controller espone un dizionario di statistiche aggregato nel CSV della run. Per i test dei primi sei capitoli vengono utilizzati:

- numero di switch decisi dal margine e numero di switch forzati dal massimo verde;
- passi in cui `Nmin` ha mantenuto la fase;
- eventi di blocco, rilascio e passi con spillback attivo;
- passi di estensione dovuti a un plotone;
- numero e somma dei bonus di fairness positivi.

Queste misure non sono obiettivi da massimizzare. Un numero elevato di attivazioni può indicare che la regola affronta un fenomeno frequente, ma anche che interviene troppo aggressivamente. La loro funzione è spiegare le metriche di mobilità e verificare la coerenza tra configurazione e comportamento.

5 Metodologia e piattaforma sperimentale

La qualità di una politica semaforica non può essere stabilita da una singola traccia. Il protocollo deve controllare domanda, seed, durata e completezza dei viaggi, oltre a rendere confrontabili gli output. Il capitolo definisce la pipeline di ablation, i modelli di domanda, le metriche e le minacce alla validità. La configurazione specifica di Manhattan è presentata nel capitolo 6.

5.1 Obiettivi della valutazione

La valutazione risponde a quattro domande:

1. il Max-Pressure riduce attesa e tempo di viaggio rispetto al programma statico nativo?
2. il vantaggio rimane osservabile al crescere della domanda?
3. quali estensioni migliorano il controller base e in quale regime?
4. il comportamento interno conferma che la regola specialistica si è effettivamente attivata?

Per rispondere, si adotta un confronto per ablation: ogni scenario abilita una sola famiglia di modifiche rispetto a `mp_base`. Le popolazioni sono identiche tra controller e cambiano soltanto tra seed o livello di domanda. Il piano fisso `fixed_program0` funge da riferimento esterno, mentre `mp_base` permette di misurare il contributo marginale delle estensioni.

5.2 Pipeline di ablation

`utils/run_ablation.py` automatizza la campagna. Dati insieme di mappe, domande, distribuzioni, seed e scenari, costruisce tutti i casi, prepara gli input, esegue i processi e aggrega gli output. Questa automazione è stata necessaria perché comandi

manuali lunghi avevano prodotto, nelle prime prove, errori difficili da diagnosticare e risultati non sempre omogenei.

5.2.1 Generazione dei casi sperimentali

Il numero totale di casi è

$$N_{case} = N_{mappe} N_{set} N_{domande} N_{seed} N_{scenari}. \quad (5.1)$$

Prima dell'avvio, la pipeline risolve i nomi delle mappe e degli scenari, calcola i preset di domanda effettivi e scrive la configurazione. Per ciascuna combinazione di mappa, set, domanda e seed genera una sola popolazione. Tutti i controller riferiti a quella combinazione ricevono lo stesso file.

Gli scenari sono raccolti in pack versionati nel codice. Il pack contiene nomi interpretabili e flag completi; il report non deve quindi inferire a posteriori quali parametri fossero stati usati.

5.2.2 Esecuzione parallela

I casi sono indipendenti e vengono assegnati a un pool di worker. Ogni worker avvia un processo SUMO e un processo Python di controllo. Il parametro `-jobs` limita il numero di casi simultanei: aumentarne il valore riduce il tempo totale finché CPU, memoria e input/output non diventano il collo di bottiglia.

Il parallelismo è tra simulazioni, non all'interno del Max-Pressure. Questa scelta è adatta a una matrice di molti casi e mantiene deterministico il controller di ciascuna simulazione. Le directory isolate impediscono che due worker sovrascrivano `tripinfo`, log o metriche.

5.2.3 Organizzazione delle run

Ogni campagna riceve un identificativo del tipo `run_NNNN_AAAAMMG_hhmmss`. La directory contiene popolazioni, risultati aggregati e una sottodirectory per ciascun caso. Il file di progresso riporta il totale pianificato e il conteggio di successi e fallimenti.

Questa organizzazione preserva la provenienza dei risultati e permette di confrontare campagne successive senza mescolarne i file. I report finali usati nel capitolo 6 provengono interamente da una sola run completa.

5.2.4 Report automatici

Per ogni gruppo mappa-set-domanda-scenario vengono calcolate la media sui seed e la deviazione standard campionaria. Se x_s è la metrica del seed s ,

$$\bar{x} = \frac{1}{n} \sum_{s=1}^n x_s, \quad (5.2)$$

$$s_x = \sqrt{\frac{1}{n-1} \sum_{s=1}^n (x_s - \bar{x})^2}. \quad (5.3)$$

La differenza percentuale rispetto alla baseline b è

$$\Delta_x = 100 \frac{\bar{x} - \bar{x}_b}{\bar{x}_b}. \quad (5.4)$$

Per attesa e tempo di viaggio un valore negativo indica un miglioramento. Le classifiche usano come criterio primario il tempo medio di attesa, ma la conclusione viene controllata sulle altre metriche e sulla censura.

Quando si confrontano due controller sugli stessi seed, si considerano anche le differenze appaiate $d_s = x_{b,s} - x_{c,s}$. L'intervallo di confidenza al 95% è

$$\bar{d} \pm t_{0,975,n-1} \frac{s_d}{\sqrt{n}}. \quad (5.5)$$

Con soli cinque seed l'intervallo non sostituisce una campagna statistica più ampia, ma aiuta a distinguere miglioramenti ampi e stabili da differenze di pochi decimi di secondo.

5.3 Generazione della domanda di traffico

Una popolazione contiene identificativo, route, istante di partenza e tipo di ogni veicolo. La finestra di generazione è distinta dalla durata totale della simulazione: dopo

l'ultima partenza la simulazione prosegue finché tutti i veicoli completano il viaggio, salvo un limite esplicito.

I tipi sono estratti con probabilità 75% autovetture, 10% veicoli per consegne, 10% motocicli, 3% camion e 2% autobus. L'eterogeneità influenza lunghezza, accelerazione, distanza minima e dinamica di partenza, rendendo la popolazione meno artificiale di un insieme composto da sole autovetture.

5.3.1 Livelli low, medium e high

I nomi low, medium e high non corrispondono a valori universali. La pipeline può applicare override per mappe note o una regola euristica basata sul numero di route. Per reti con oltre 650 route la domanda media è 4000 veicoli; low è il 60% e high il 140%, cioè rispettivamente 2400 e 5600 veicoli. Tutte le partenze sono collocate nell'intervallo $[0, 3600]$ secondi.

Questa calibrazione rende i livelli proporzionati alla varietà di percorsi della mappa. Non garantisce però che “high” produca saturazione o spillback: la gravità effettiva dipende anche da capacità, lunghezza delle route e concentrazione spaziale. Nel capitolo dei risultati tale distinzione risulta essenziale.

5.3.2 Distribuzione balanced

Nel set balanced ogni route ha uguale probabilità di essere scelta e gli istanti di partenza sono uniformi nella finestra. È la configurazione usata per la campagna Manhattan: distribuisce la domanda sull'intera griglia e permette di osservare il comportamento generale del controller senza introdurre corridoi caldi deliberati.

Balanced non significa che ogni arco riceva esattamente lo stesso flusso. Le route hanno lunghezze e sovrapposizioni differenti; significa che il generatore non applica pesi aggiuntivi alla loro selezione.

5.3.3 Distribuzione skewed

Nel set skewed la probabilità di una route è proporzionale a

$$w_r = E_r^{1,4}, \quad (5.6)$$

in cui E_r è il numero di archi della route. Le partenze rimangono uniformi. Le route più lunghe vengono quindi campionate più spesso e, condividendo più segmenti, tendono a creare assi maggiormente carichi.

La scelta delle singole route non è fissata manualmente a ogni esecuzione: è casuale ma riproducibile tramite seed. Si controlla la distribuzione, non la lista esatta dei percorsi più trafficati. Questo set è stato introdotto per le campagne sulla rete reale e non compare nel confronto Manhattan del capitolo 6.

5.3.4 Distribuzione peak

Peak usa lo stesso campionamento spaziale di skewed e concentra anche le partenze. Il generatore estrae da due distribuzioni beta ampie, una anticipata e una posticipata nella finestra, quindi miscela il valore con un'estrazione uniforme:

$$u_{peak} = 0,80 u_{\beta} + 0,20 u_{uniforme}. \quad (5.7)$$

La scelta della prima componente avviene con probabilità 0,55. Il risultato simula due ondate di domanda senza concentrare tutti i veicoli nello stesso istante. Anche questa distribuzione è riproducibile e viene usata nella parte successiva della tesi dedicata a Bologna.

5.3.5 Seed e riproducibilità

Un unico oggetto pseudo-casuale, inizializzato dal seed, determina partenze, route e tipi. La popolazione viene poi ordinata per istante di partenza e serializzata. Nella campagna Manhattan si usano i seed da 1 a 5. Poiché ogni file è condiviso da tutti gli scenari, le differenze tra controller sono appaiate e non dipendono da un diverso campione di veicoli.

5.4 Profilo di guida human-light

Il profilo `human_light` riduce l'idealizzazione del comportamento senza introdurre un tuning aggressivo. La velocità iniziale usa il valore `desired`, non sempre il massimo consentito. Per ciascun tipo vengono specificati tempo di reazione τ , imperfezione di guida σ , intervallo di azione, gap alle precedenze e ritardo di partenza.

Tabella 5.1. Parametri principali del profilo *human_light*.

Tipo	τ [s]	σ	Azione [s]	Gap [s]	Startup [s]
Autovettura	1,45	0,55	1,0	1,40	0,80
Consegna	1,55	0,50	1,0	1,45	1,00
Camion	1,75	0,45	1,0	1,60	1,20
Autobus	1,75	0,45	1,0	1,60	1,20
Motociclo	1,25	0,65	1,0	1,25	0,45

Il profilo è identico per tutti i controller. Non cerca di riprodurre una specifica popolazione cittadina, ma evita partenze istantanee e reazioni perfette che potrebbero favorire irrealisticamente switch molto frequenti.

5.5 Controller di confronto

5.5.1 Fixed program0

`fixed_program0` seleziona esplicitamente il programma con identificativo `0` e lascia a SUMO l'esecuzione ciclica delle durate definite nella mappa. Non osserva code e non modifica il piano. È la baseline principale di Manhattan perché rappresenta il funzionamento semaforico nativo disponibile senza controllo adattivo.

5.5.2 Fixed tuned

La piattaforma supporta anche `fixed_tuned`, che forza a 30 secondi le fasi principali del `program0` mantenendo le transizioni. Questo baseline è stato utile nelle campagne esplorative su Bologna, ma non viene incluso nella campagna Manhattan finale: il suo parametro non deriva da un'ottimizzazione specifica della griglia e avrebbe aggiunto un confronto non necessario al primo studio per ablation.

5.5.3 Max-Pressure di base

`mp_base` usa pressione upstream–downstream, minimo verde di 10 secondi, massimo verde di 120 secondi e margine nullo. Nessuna estensione è abilitata. Esso ha due ruoli: controller adattivo generale e riferimento interno per stabilire se una modifica specialistica aggiunga valore.

5.6 Metriche

5.6.1 Tempo di attesa

Per ciascun veicolo SUMO fornisce il tempo totale trascorso in attesa. La media è l'indicatore primario perché misura direttamente la conseguenza delle scelte semaforiche. Viene riportato anche il 95-esimo percentile, utile a verificare che una buona media non nasconda una coda di utenti fortemente penalizzati.

5.6.2 Tempo di viaggio e time loss

Il tempo di viaggio è la differenza tra arrivo e partenza. La *time loss* è la perdita rispetto a una percorrenza ideale secondo il modello SUMO e comprende rallentamenti, arresti e interazioni. Le due metriche non sono intercambiabili: route più lunghe aumentano il viaggio anche a parità di controllo, mentre il time loss isola meglio l'inefficienza incontrata.

5.6.3 Velocità media

Per ogni viaggio completato la velocità media è il rapporto tra distanza e durata; il report aggrega poi i veicoli. Un controller efficace dovrebbe produrre, a parità di route, velocità maggiori coerentemente con minori attese e perdite di tempo.

5.6.4 Viaggi completati e throughput

Il throughput osservato è il numero di viaggi completati. Quando la simulazione prosegue fino a svuotamento, tutti gli scenari dovrebbero completare la stessa popolazione. In presenza di un limite temporale, il numero di arrivi diventa invece una metrica essenziale: un controller che lascia veicoli bloccati non può essere valutato soltanto sui pochi arrivati.

5.6.5 Censura

Siano N_p i viaggi pianificati e N_c quelli completati. La censura è

$$C = \frac{N_p - N_c}{N_p}. \quad (5.8)$$

Una censura positiva indica che le metriche di viaggio sono calcolate su un sottoinsieme. Non equivale automaticamente a un errore, ma rende il confronto più delicato e deve essere riportata. La campagna Manhattan usa `max_steps=0`, cioè nessun cutoff imposto dalla pipeline, e termina con $C = 0$ in tutti i 135 casi.

5.7 Criteri di confronto

Il criterio primario è la minore attesa media, a condizione che la censura sia comparabile e preferibilmente nulla. Si verificano poi tempo di viaggio, time loss, percentile 95 e velocità. Una variante è considerata sostanzialmente migliore del base se il vantaggio è coerente sui seed e non deriva da un numero inferiore di viaggi completati.

Le attivazioni interne vengono usate come evidenza causale debole ma utile. Ad esempio, risultati e contatori identici tra spillback e base indicano che la soglia non è stata raggiunta. Non si conclude che lo spillback sia inutile in generale; si conclude che la campagna non ha testato il regime per cui è stato progettato.

5.8 Ambiente hardware e software

La campagna finale Manhattan è stata eseguita sul server `pccabri2` del dominio `mat.unimo.it`. L'ambiente comprendeva Ubuntu 22.04.5 LTS, kernel Linux 5.15, processore AMD Athlon II X4 620 con quattro core, circa 11 GiB di RAM, Python 3.10.12 e binario SUMO 1.27.0. L'ambiente virtuale conteneva TraCI e sumolib 1.20.0, PyYAML 6.0.1 e NumPy 1.26.4.

Sono stati usati tre worker paralleli. I 135 casi hanno richiesto circa 7 ore e 20 minuti di tempo reale. La GPU non è coinvolta: SUMO e il controller sono principalmente CPU-bound, e il parallelismo utile deriva dall'esecuzione di più processi indipendenti.

5.9 Minacce alla validità

Validità interna. Gli scenari condividono popolazioni e codice di supporto, ma parametri differenti possono modificare più di un meccanismo. Il caso `Nmin`, che riduce anche il minimo verde, viene perciò interpretato con i contatori. Errori del simulatore o della mappa sono mitigati da preflight, log, test automatici e controllo degli errori SUMO.

Validità di costruito. Attesa, viaggio e time loss rappresentano aspetti diversi dell'efficienza, ma non descrivono sicurezza, priorità del trasporto pubblico o esperienza pedonale. Le conclusioni sono limitate alla mobilità veicolare. Indicatori ambientali eventualmente presenti negli output generici non appartengono al modello di valutazione.

Validità esterna. Manhattan è regolare, completamente semaforizzata e usa domanda balanced. I risultati non si trasferiscono automaticamente a una rete reale irregolare o a traffico fortemente concentrato. La seconda parte della tesi affronta questa limitazione con Bologna e con i set skewed e peak.

Validità statistica. Cinque seed permettono di osservare stabilità e costruire confronti appaiati, ma non sono sufficienti per caratterizzare ogni coda della distribuzione. Differenze molto piccole tra varianti devono quindi essere considerate indicative, non definitive. Al contrario, riduzioni di decine di secondi rispetto al fixed risultano molto maggiori della variabilità tra seed.

6 Valutazione su Manhattan 8x8

La rete Manhattan costituisce il primo banco di prova completo perché riduce le peculiarità topologiche e consente di osservare il comportamento dei controller in una griglia ripetibile. Il capitolo presenta configurazione, risultati e contatori della run finale. L'obiettivo non è scegliere un unico parametro universale, ma capire quali meccanismi producano un beneficio prima di passare alla rete reale.

6.1 Motivazione dello scenario sintetico

Una rete reale combina contemporaneamente segmenti corti, svolte, incroci con fasi diverse e domanda spazialmente non uniforme. Se una variante peggiora, può essere difficile stabilire quale caratteristica abbia causato il risultato. Manhattan 8×8 offre invece incroci quasi omogenei, elevata semaforizzazione e molte route. È quindi adatta a una ablation controllata.

La regolarità non rende il test banale. Sessanta semafori prendono decisioni locali e le code si propagano attraverso la griglia. Il piano fisso, inoltre, non usa offset coordinati: tutti i programmi partono con offset zero. La campagna misura dunque quanto l'adattamento locale riesca a sfruttare una domanda diffusa rispetto alla ripetizione dello stesso ciclo.

6.2 Topologia della rete

La mappa `manhattan8x8_100pc` contiene 64 giunzioni non interne, delle quali 60 semaforizzate e quattro angoli regolati a priorità. I 224 archi non interni hanno complessivamente 448 corsie, due per arco. Le lunghezze delle corsie sono comprese tra 139,2 e 143,2 metri, con media circa 139,5 metri. Il file delle route contiene 800 percorsi, da 1 a 15 archi e con lunghezza media pari a 7,276 archi.

La figura 6.1 rappresenta schematicamente la griglia. I nodi pieni indicano intersezioni semaforizzate e i quattro quadrati bianchi gli angoli a priorità.

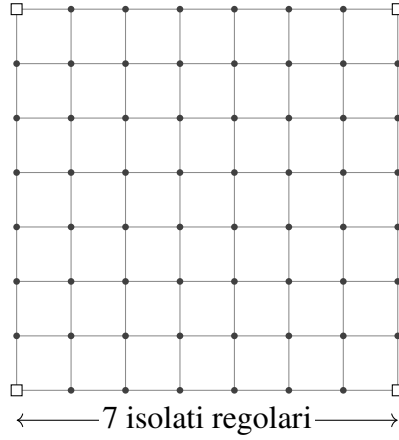


Figura 6.1. Schema della rete Manhattan 8×8 .

Ogni semaforo possiede due logiche. Il program0, usato dal fixed, ha due fasi verdi da 42 secondi. Ciascun passaggio comprende 3 secondi di giallo e 3 secondi di all-red, per un ciclo complessivo di 96 secondi. La logica dinamica mantiene le stesse transizioni ma assegna alle fasi principali una durata sufficientemente lunga da lasciare al controller esterno la decisione del cambio. In entrambi i casi le transizioni di sicurezza sono definite nella mappa e preservate.

6.3 Configurazione sperimentale

La run finale è identificata da `run_0023_20260612_131724`. Comprende una mappa, una distribuzione balanced, tre domande, cinque seed e nove scenari:

$$1 \times 1 \times 3 \times 5 \times 9 = 135 \text{ simulazioni.} \quad (6.1)$$

Le partenze sono distribuite uniformemente nei primi 3600 secondi. Low contiene 2400 veicoli, medium 4000 e high 5600. Non è imposto un numero massimo di passi: ciascun caso prosegue fino a esaurimento. Tutti i casi sono terminati con successo; in ciascuno il numero di viaggi completati coincide con quello pianificato e la censura è zero. Non sono stati rilevati nei log avvisi di collisione, teletrasporto o frenata d'emergenza.

La tabella 6.1 riassume gli scenari. I valori non sono tutte le combinazioni provate durante lo sviluppo, ma una configurazione definitiva per ogni famiglia.

Tabella 6.1. Scenari della campagna Manhattan.

Scenario	Configurazione caratterizzante
fixed_program0	Programma statico 0 della mappa
mp_base	Minimo 10 s, massimo 120 s, margine nullo
mp_lta_g060_sf050	$\gamma = 0,60$, $s = 0,50$ veicoli/s
mp_nmin_a080_f2	$\alpha = 0,80$, floor 2, minimo 4 s, $g = 0,25$
mp_downstream_b04	$\beta = 0,4$, EMA $\eta = 0,30$
mp_spillback_on90_off80	Soglie 0,90/0,80, almeno 3 veicoli fermi
mp_fair_mu5_w20	$\mu = 5$, $w_{1/2} = 20$ s
mp_platoon_x4	Headway e gap 2,1 s, estensione massima 4 s
mp_switch_rel005	Margine relativo 5%

6.4 Confronto tra Max-Pressure e controllo fisso

La tabella 6.2 riporta il tempo medio di attesa e la deviazione standard tra seed. Tutte le varianti Max-Pressure riducono nettamente l'attesa rispetto al fixed in ciascun livello. Anche la variante meno favorevole rimane molto distante dalla baseline.

Tabella 6.2. Tempo medio di attesa su Manhattan, media $\pm$ deviazione standard sui cinque seed, in secondi.

Scenario	Low	Medium	High
fixed_program0	65,85 $\pm$ 1,11	66,89 $\pm$ 0,24	67,75 $\pm$ 0,46
mp_base	26,69 $\pm$ 0,27	29,48 $\pm$ 0,35	32,07 $\pm$ 0,33
mp_lta_g060_sf050	30,56 $\pm$ 0,46	31,45 $\pm$ 0,57	32,77 $\pm$ 0,45
mp_nmin_a080_f2	23,54 $\pm$ 0,41	26,70 $\pm$ 0,15	32,03 $\pm$ 0,76
mp_downstream_b04	31,76 $\pm$ 0,41	33,05 $\pm$ 0,66	34,71 $\pm$ 0,61
mp_spillback_on90_off80	26,69 $\pm$ 0,27	29,48 $\pm$ 0,35	32,07 $\pm$ 0,33
mp_fair_mu5_w20	26,60 $\pm$ 0,31	29,63 $\pm$ 0,35	31,81 $\pm$ 0,10
mp_platoon_x4	26,92 $\pm$ 0,30	29,43 $\pm$ 0,49	31,94 $\pm$ 0,33
mp_switch_rel005	26,69 $\pm$ 0,27	29,48 $\pm$ 0,35	32,07 $\pm$ 0,33

Il vincitore low e medium è Nmin; in high il valore minimo appartiene alla fairness. Rispetto al fixed, Nmin riduce l'attesa del 64,26% in low e del 60,09% in medium. In high la fairness ottiene una riduzione del 53,05%. La figura 6.2 mostra che il divario rispetto al

fixed rimane ampio al crescere della domanda, mentre le migliori varianti MP convergono verso valori simili.

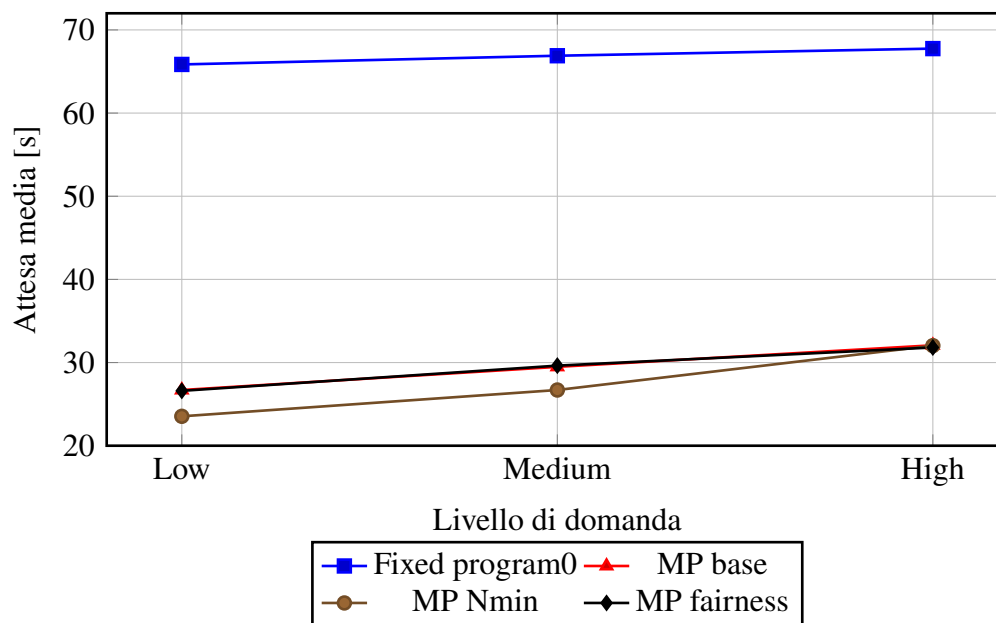


Figura 6.2. Andamento del tempo medio di attesa per controller selezionati.

Il vantaggio non riguarda soltanto la media. In low il 95-esimo percentile dell'attesa passa da 137,22 secondi del fixed a 48,00 di Nmin; in medium da 138,80 a 54,20. In high la fairness ottiene 66,00 secondi contro 141,00 del fixed. La riduzione interessa quindi anche i viaggi maggiormente penalizzati.

Tabella 6.3. Confronto tra vincitore e fixed per livello di domanda.

Domanda	Vincitore	Attesa [s]	Δ attesa [%]	Viaggio [s]	Δ viaggio [%]	Time loss [s]	Velocità [m/s]
Low	Nmin	23,54	-64,26	137,47	-22,04	54,58	8,33
Medium	Nmin	26,70	-60,09	144,78	-19,81	61,57	7,94
High	Fairness	31,81	-53,05	152,23	-17,44	68,91	7,58
Low fixed	–	65,85	–	176,33	–	93,42	6,74
Medium fixed	–	66,89	–	180,56	–	97,32	6,58
High fixed	–	67,75	–	184,39	–	101,06	6,43

Le differenze appaiate confermano che il confronto con il fixed è robusto. Il risparmio medio di attesa è 42,31 secondi in low per Nmin, con intervallo al 95%

[41,16; 43,47]; 40,20 secondi in medium, intervallo [39,92; 40,47]; 35,94 secondi in high per fairness, intervallo [35,30; 36,58]. La variabilità tra seed è dunque piccola rispetto al guadagno.

6.5 Valutazione delle singole estensioni

Il confronto con il fixed dimostra il valore del controllo adattivo nella griglia, ma non implica che ogni estensione migliori `mp_base`. Le sezioni seguenti separano i due livelli di confronto.

6.5.1 Lost-Time-Aware

LTA riduce il numero di switch decisi dal punteggio: da 6071 a 5273 in low, da 8591 a 7566 in medium e da 10184 a 9106 in high, come media sui seed. Tuttavia l'attesa aumenta rispettivamente di 3,86, 1,97 e 0,70 secondi rispetto al base. Gli interventi del massimo verde crescono fortemente in low, da 91 a circa 260.

Il risultato indica che il margine compensa troppo aggressivamente il costo di transizione nella domanda leggera e ritarda cambi utili. Con traffico elevato la penalizzazione si riduce, perché le pressioni più grandi superano più facilmente il margine. LTA non è quindi errata in principio, ma la sua configurazione non è vantaggiosa nella griglia balanced studiata.

6.5.2 Nmin dinamico

Nmin produce il miglior risultato low e medium. Rispetto a `mp_base` risparmia 3,16 secondi in low, intervallo appaiato al 95% [2,55; 3,77], e 2,78 secondi in medium, intervallo [2,50; 3,07]. In high il vantaggio è soltanto 0,04 secondi, con intervallo [-0,68; 0,76], quindi non distinguibile dalla variabilità osservata.

I passi di hold Nmin sono in media 0 in low, 2,4 in medium e 23,2 in high. Il forte vantaggio a basso traffico non è quindi causato dalla permanenza dinamica, che non si attiva, ma soprattutto dal minimo verde ridotto a 4 secondi. Questa configurazione consente di correggere più rapidamente una fase che ha già esaurito la domanda. In high l'hold entra in funzione, ma il tempo di viaggio sale a 155,28 secondi contro 152,59 del

base: proteggere il servizio della fase corrente può ridurre l'attesa media senza migliorare uniformemente l'intero viaggio.

6.5.3 Fairness

La fairness è quasi neutra in low e medium e nominalmente migliore in high. I bonus positivi sono applicati in media 5584, 7951 e 9538 volte nei tre livelli; il meccanismo è quindi attivo. All'aumentare del carico, più fasi conservano domanda mentre non sono servite e il bonus può correggere squilibri persistenti.

In high il vantaggio sul base è 0,26 secondi. L'intervallo appaiato al 95% del risparmio è $[-0,08; 0,60]$ e include zero. Inoltre, considerando il vincitore di ogni singolo seed tra le varianti di testa, fairness vince due seed, Nmin due e platoon uno. È corretto definirla vincitrice nominale aggregata, non una superiorità statistica definitiva.

6.5.4 Penalità downstream

La penalità downstream è la variante peggiore tra gli MP in tutti i livelli. Rispetto al base aumenta l'attesa di 5,06 secondi in low, 3,57 in medium e 2,64 in high. Il numero di decisioni di switch cresce, mentre gli interventi del massimo verde quasi scompaiono.

La rete non raggiunge un regime in cui le uscite siano prossime al blocco. La penalità continua finisce quindi per reagire a normali livelli di occupazione, modificando frequentemente la classifica delle fasi senza prevenire uno spillback reale. Il costo relativo diminuisce in high, ma rimane osservabile anche su viaggio, time loss e velocità.

6.5.5 Anti-spillback

Lo scenario anti-spillback produce valori identici al base fino alla precisione dei report. Tutti i contatori di blocco, rilascio e passi bloccati sono zero per ogni domanda. La spiegazione non è che il codice non venga eseguito: nessuna corsia raggiunge contemporaneamente il punteggio 0,90 e almeno tre veicoli fermi richiesti per l'attivazione.

Il risultato delimita la campagna. Anche 5600 veicoli balanced costituiscono un carico elevato per il confronto, ma non una condizione di spillback. Per valutare la protezione sono necessarie route più concentrate, picchi temporali, corsie ricevanti corte o una domanda maggiore. La successiva introduzione dei set skewed e peak nasce precisamente da questa osservazione.

6.5.6 Platoon extension

I prolungamenti per plotone aumentano da 113,6 passi medi in low a 227 in medium e 288,8 in high. Il meccanismo rileva quindi arrivi compatti. Le metriche restano però molto vicine al base: leggermente peggiori in low, migliori di 0,06 secondi in medium e di 0,13 in high.

L'estensione massima di 4 secondi è prudente e il traffico balanced non produce corridoi dominanti. Inoltre, nello scenario standalone non è attivo un guard downstream effettivo. Il test mostra che il riconoscimento dei plotoni funziona, ma non dimostra un vantaggio sostanziale nella griglia uniforme.

6.5.7 Margini di switch

Il margine relativo del 5% produce risultati e contatori esattamente uguali al base. In tutti i casi in cui una fase alternativa supera quella corrente, il divario è già maggiore del margine, oppure il cambio non sarebbe avvenuto nemmeno con margine nullo. La banda morta non modifica quindi alcuna decisione nella campagna.

Questo esito è informativo: la soglia non va considerata stabilizzante soltanto perché è stata abilitata. Per avere effetto dovrebbe essere aumentata o testata in uno scenario con pressioni più vicine e oscillanti; un valore più alto, tuttavia, rischierebbe di riprodurre il problema osservato con LTA.

6.6 Analisi dei contatori di attivazione

La tabella 6.4 sintetizza il contatore specifico di ciascuna variante. Le unità sono passi o eventi medi per simulazione; per il base viene riportata la coppia switch da margine/interventi da massimo verde.

Due conclusioni emergono. Primo, “attivo” non significa “utile”: fairness e platoon intervengono spesso, ma il vantaggio sul base è piccolo; downstream modifica molte decisioni e peggiora. Secondo, valori finali uguali possono essere spiegati: spillback e margine relativo non alterano il percorso decisionale, mentre la coincidenza non è dovuta a un problema di aggregazione.

Tabella 6.4. Contatori medi di attivazione per run.

Meccanismo	Low	Medium	High
Base: switch / max-green	6071 / 91	8591 / 40	10184 / 21
LTA: switch / max-green	5273 / 260	7566 / 158	9106 / 90
Nmin: passi di hold	0,0	2,4	23,2
Fairness: bonus positivi	5584	7951	9538
Spillback: passi bloccati	0,0	0,0	0,0
Platoon: passi estesi	113,6	227,0	288,8

6.7 Confronto complessivo

La campagna sostiene con chiarezza l'ipotesi principale: su Manhattan 8×8 balanced, il controllo Max-Pressure riduce in modo ampio attesa, viaggio e time loss rispetto al program0 nativo, mantenendo censura nulla. Il Max-Pressure base da solo riduce l'attesa del 59,46%, 55,93% e 52,66% nei tre livelli. Non è quindi necessario un modulo specialistico per ottenere il beneficio fondamentale.

Tra le estensioni, Nmin è la scelta più efficace a domanda bassa e media, principalmente grazie al minimo verde più corto. A domanda alta fairness, platoon, Nmin e base sono raccolti entro 0,26 secondi di attesa. La classifica nominale non deve oscurare tale prossimità. LTA e downstream mostrano invece che aggiungere prudenza o informazione non produce automaticamente una politica migliore: una regola utile in congestione può costare capacità quando il fenomeno bersaglio è assente.

Il piano fixed rimane quasi costante tra 65,85 e 67,75 secondi di attesa. Questo non significa che il traffico non aumenti: viaggio e time loss peggiorano e la velocità scende. Significa che il ciclo statico impone una componente dominante di ritardo anche in low, mentre il Max-Pressure evita verdi assegnati a movimenti con poca domanda. La rete regolare e l'assenza di offset coordinati favoriscono questo vantaggio locale. Non si può estendere la conclusione a un piano fisso ottimizzato per ogni profilo di domanda.

6.8 Indicazioni ricavate per lo scenario reale

La valutazione Manhattan produce cinque indicazioni per Bologna:

1. mantenere mp_base come riferimento, perché è già robusto e ogni specialista deve

- giustificare il proprio costo;
2. ridurre il minimo verde può essere molto efficace a basso carico, ma va separato concettualmente dall'hold dinamico N_{min} ;
 3. usare i contatori per verificare l'attivazione: senza congestione downstream, lo spillback non può essere giudicato;
 4. introdurre domanda spazialmente e temporalmente non uniforme per creare condizioni più realistiche di corridoio e picco;
 5. evitare l'idea di una variante universalmente migliore: la quasi parità in high suggerisce di selezionare la modalità in funzione del regime locale invece di mantenere sempre la stessa estensione.

Queste indicazioni motivano il passaggio dalla griglia regolare alla rete di Bologna. In quella mappa il programma statico è più competitivo, la topologia è eterogenea e il costo degli switch assume un peso maggiore. Il problema non è più soltanto dimostrare che il Max-Pressure possa funzionare, ma stabilire quando convenga comportarsi in modo adattivo, quando conservare il piano nativo e quale specialista attivare.

Riferimenti bibliografici

- [Beh+11] Michael Behrisch et al. «SUMO – Simulation of Urban MObility: An Overview». In: *Proceedings of the Third International Conference on Advances in System Simulation*. 2011, pp. 63–68. URL: https://sumo.dlr.de/pdf/simul_2011_3_40_50150.pdf (visitato il 13/06/2026).
- [CF12] Burak Cesme e Peter G. Furth. «Multiheadway Gap-Out Logic for Actuated Control on Multilane Approaches». In: *Transportation Research Record* 2311.1 (2012), pp. 117–123. DOI: 10.3141/2311-11.
- [Ecl26a] Eclipse SUMO Project. *SUMO Documentation*. 2026. URL: <https://sumo.dlr.de/docs/> (visitato il 13/06/2026).
- [Ecl26b] Eclipse SUMO Project. *TraCI Documentation*. 2026. URL: <https://sumo.dlr.de/docs/TraCI.html> (visitato il 13/06/2026).
- [Gar83] Nathan H. Gartner. «OPAC: A Demand-Responsive Strategy for Traffic Signal Control». In: *Transportation Research Record* 906 (1983), pp. 75–81.
- [Gre+15] Julien Gregoire et al. «Capacity-Aware Backpressure Traffic Signal Control». In: *IEEE Transactions on Control of Network Systems* 2.2 (2015), pp. 164–173. DOI: 10.1109/TCNS.2014.2378871.
- [Le+15] Tien Mai Le et al. «Decentralized Signal Control for Urban Road Networks». In: *Transportation Research Part C: Emerging Technologies* 58 (2015), pp. 431–450. DOI: 10.1016/j.trc.2014.11.009.
- [Lev23] Michael W. Levin. «Max-Pressure Traffic Signal Timing: A Summary of Methodological and Experimental Results». In: *Journal of Transportation Engineering, Part A: Systems* 149.4 (2023), p. 03123001. DOI: 10.1061/JTEPBS.TEENG-7578.
- [MUH20] Pablo Mercader, Wajdi Uwayid e Jack Haddad. «Max-Pressure Traffic Controller Based on Travel Times: An Experimental Analysis». In: *Transportation Research Part C: Emerging Technologies* 110 (2020), pp. 275–290. DOI: 10.1016/j.trc.2019.10.002.

- [Noa+21] Mohammad Noaeen et al. «Real-Time Decentralized Traffic Signal Control for Congested Urban Networks Considering Queue Spillbacks». In: *Transportation Research Part C: Emerging Technologies* 133 (2021), p. 103407. doi: 10.1016/j.trc.2021.103407.
- [RB91] Dennis I. Robertson e Robert D. Bretherton. «Optimizing Networks of Traffic Signals in Real Time: The SCOOT Method». In: *IEEE Transactions on Vehicular Technology* 40.1 (1991), pp. 11–15.
- [SY18] Xiaolei Sun e Yafeng Yin. «A Simulation Study on Max Pressure Control of Signalized Intersections». In: *Transportation Research Record* 2672.18 (2018), pp. 117–127. doi: 10.1177/0361198118786840.
- [Var13] Pravin Varaiya. «Max Pressure Control of a Network of Signalized Intersections». In: *Transportation Research Part C: Emerging Technologies* 36 (2013), pp. 177–195. doi: 10.1016/j.trc.2013.08.014.
- [Wan+22] Xu Wang et al. «Learning the Max Pressure Control for Urban Traffic Networks Considering the Phase Switching Loss». In: *Transportation Research Part C: Emerging Technologies* 140 (2022), p. 103670. doi: 10.1016/j.trc.2022.103670.
- [Weg+08] Axel Wegener et al. «TraCI: An Interface for Coupling Road Traffic and Network Simulators». In: *Proceedings of the 11th Communications and Networking Simulation Symposium*. 2008, pp. 155–163. doi: 10.1145/1400713.1400740.
- [Won+12] Tichakorn Wongpiromsarn et al. «Distributed Traffic Signal Control for Maximum Network Throughput». In: *Proceedings of the 15th International IEEE Conference on Intelligent Transportation Systems*. 2012, pp. 588–595. doi: 10.1109/ITSC.2012.6338817.
- [WLG18] Xiaoguang Wu, Henry X. Liu e Douglas Gettman. «Delay-Based Traffic Signal Control for Throughput Optimality and Fairness at an Isolated Intersection». In: *IEEE Transactions on Vehicular Technology* 67.2 (2018), pp. 896–909. doi: 10.1109/TVT.2017.2760820.